TRANSURBAN — explicação da API, sem slides

Apresentação de 20/10/2026. Duração sugerida: 8 a 10 minutos. Usar somente Swagger e VS Code, sem slides. Equipe confirmada pelo usuário: José Henrique, Bruno, Kauan, Gustavo, Jullyany, Camila e Ana Carolina. A divisão abaixo é de fala e demonstração; não atribui autoria individual do código.

Ordem	Integrante	Parte	Tempo
1	Bruno	Objetivo e papel da API	40 s
2	Kauan	Arquivos, PK, FK e relacionamento	1 min 30 s
3	José Henrique	POST/GET de cidades e linhas	2 min
4	Gustavo	Erros 404 e 422; consulta após rejeição	1 min 30 s
5	Jullyany	CSV, unidade de análise e qualidade	1 min
6	Camila	Cálculos, agrupamentos e interpretação	1 min
7	Ana Carolina	Evidências, GitHub, limites e encerramento	40 s

Antes da aula: preparar, não improvisar

O teste escolar já demonstrou POST/GET de cidades e linhas, cidade relacionada no JSON, 404 sem inserção e 422 para nome vazio. A versão 1.1 acrescenta IA e `/health`; é preciso atualizar e iniciar essa versão na máquina antes de demonstrar as rotas novas. A atualização na nuvem não altera automaticamente o computador da escola.

Hoje, pare o servidor antigo com Ctrl+C e siga `backend_escola/FINALIZAR_ENTREGA.md` para atualizar. Faça um ensaio. Na apresentação, com dependências já instaladas, abra o terminal na pasta `backend_escola` e use:

```
./.venv-escola/bin/python -m uvicorn main:app --host 127.0.0.1 --port 8000
```

Deixe o terminal aberto. Abra `http://127.0.0.1:8000/docs` no mesmo computador. Deixe VS Code com `database.py`, `models.py`, `schemas.py` e `main.py` disponíveis. Não abra `.env`. Não é necessário iniciar o frontend Vite nem abrir slides.

Parte 1 — Bruno: explicar o objetivo (40 segundos)

Fala sugerida:

“Nosso projeto é o TRANSURBAN, voltado ao transporte entre Maringá e Sarandi. Nesta avaliação, vamos demonstrar a API: a parte do sistema que recebe pedidos, valida os dados, consulta ou grava no banco e devolve respostas em JSON. Adaptamos o exemplo Categoria e Produto para Cidade e Linha de ônibus.”

“Usamos Python, FastAPI, SQLAlchemy e Pydantic. O Swagger é a ferramenta de teste que estamos mostrando; ele não é o site final para passageiros.”

Parte 2 — Kauan: explicar os arquivos e o banco (1 minuto e 30 segundos)

Mostre brevemente os arquivos no VS Code, sem editar durante a fala.

“O `database.py` configura a conexão e cria as sessões de banco. O `models.py` representa as tabelas e seus relacionamentos. O `schemas.py` valida a entrada e define o formato da resposta. O `main.py` registra as rotas e coordena as operações.”

“Cidade possui `id_cidade` como chave primária. Linha de ônibus possui `id_linha` como sua chave primária e `id_cidade` como chave estrangeira. Uma cidade pode ter várias linhas; cada linha pertence a uma cidade existente.”

“ForeignKey estabelece a referência no banco. relationship permite navegar entre os objetos Python. Por isso conseguimos retornar o nome da cidade na consulta, além do identificador armazenado.”

Parte 3 — José Henrique: demonstrar cadastro e consulta (2 minutos)

No Swagger, abra POST `/cidades`, clique em Try it out e envie:

```
{"nome":"Sarandi"}
```

“POST envia dados para cadastro. A resposta 201 informa que o registro foi criado. O banco devolveu o identificador da cidade.”

Anote o `id_cidade` realmente retornado. Em POST `/linhas`, substitua o 2 do exemplo pelo número que acabou de receber:

```
{"codigo":"DEMO-20","nome":"Linha da apresentação","id_cidade":2}
```

“Antes de salvar a linha, a API verifica se a cidade informada existe. A resposta inclui a chave estrangeira e também o nome da cidade, que é uma informação mais compreensível para o usuário.”

Execute GET `/cidades` e GET `/linhas`.

“GET faz uma consulta. O status 200 mostra que a requisição foi atendida. Os registros retornaram em uma nova requisição, demonstrando a gravação e a leitura no banco configurado.”

Evite cadastrar várias vezes por engano: esta API escolar não impõe unicidade de nome ou código. Não apresente rejeição de duplicidade como funcionalidade dela.

Parte 4 — Gustavo: demonstrar as validações (1 minuto e 30 segundos)

Confirme que o ID 999999 não existe na consulta de cidades. Em POST `/linhas`, envie:

```
{"codigo":"TESTE-ERRO","nome":"Tentativa inválida","id_cidade":999999}
```

“A API responde 404 porque a cidade não existe. A operação é interrompida antes do cadastro. A FK do banco também protege essa referência.”

Execute GET `/linhas` e mostre que TESTE-ERRO não foi inserida.

Em POST `/cidades`, envie:

```
{"nome":"   "}
```

“O schema remove os espaços e rejeita o nome vazio com 422. Assim evitamos que uma entrada inválida chegue ao cadastro.”

Resumo para dizer em voz alta: “201 é cadastro, 200 é consulta, 404 indica referência inexistente e 422 indica entrada inválida.”

Parte 5 — Jullyany e Camila: rotas de dados da versão 1.1 (2 minutos)

Esta parte exige a versão atualizada. Confirme antes da apresentação que o Swagger exibe o grupo IA — Ciência de Dados.

Jullyany — fonte, significado e qualidade (1 minuto)

Execute GET `/ia/qualidade`, depois GET `/ia/resumo`.

“Na parte de Ciência de Dados, a API usa Pandas para ler e verificar um CSV desnormalizado do projeto. Cada linha representa uma observação por trecho, data e linha de ônibus. A base é sintética: não representa a operação real da frota e não permite contar viagens completas.”

“A qualidade verifica ausências, datas, valores inválidos, duplicatas e coerência entre os tempos. Se a fonte tiver problemas, os indicadores são interrompidos com 422, sem apagar silenciosamente a fonte.”

Camila — números e interpretação (1 minuto)

Mostre os números do JSON:

• 180 observações válidas.

• Soma de 1.362 minutos por observação.

• Média de 7,5667 minutos: 1.362 dividido por 180.

• Mediana de 8 minutos.

• Maior atraso de 12 minutos.

“Média é a soma dividida pela quantidade; mediana é o valor central dos dados ordenados. O máximo de 12 minutos ocorre em sete observações, e a API preserva todos os empates.”

Execute GET `/ia/agrupamentos/linha` e `/ia/agrupamentos/data`.

“Essas rotas respondem quais linhas apresentam maiores atrasos e como os valores variam no tempo. Os agrupamentos reconciliam as mesmas 180 observações e os mesmos 1.362 minutos. A soma não representa minutos perdidos por passageiros, e os grupos não comprovam causalidade.”

Não é necessário mostrar o dashboard: o objetivo aqui é explicar as respostas da API em JSON.

Parte 6 — Ana Carolina: concluir com o estado real (40 segundos)

“Demonstramos cadastro, consulta, relacionamento e validação de erros. A API também fornece indicadores auditáveis do CSV. Os testes automatizados conferem os cálculos e situações de fonte inválida.”

“O código e as evidências estão versionados no GitHub. A demonstração é local. A conexão Supabase precisa ser comprovada separadamente, com a configuração privada e os mesmos registros no banco remoto. Não estamos apresentando GPS, login ou recarga reais nesta API.”

Abra o repositório somente se houver tempo. A branch da entrega é `entrega/escola-apresentacao-2026-10-20`. Uma branch publicada não significa que as mudanças foram incorporadas à main nem que o site foi hospedado.

Perguntas prováveis e respostas curtas

Por que guardar o ID e devolver o nome? O ID mantém a referência entre tabelas; o nome facilita a leitura pelo consumidor da API.

Model e schema são iguais? Model mapeia a tabela e o relacionamento; schema valida os dados HTTP e define a resposta.

O que fazem add, commit e refresh? Add inclui o objeto na sessão; commit confirma a transação; refresh relê os valores salvos e gerados.

Por que fechar a sessão? Para liberar recursos usados pela requisição, inclusive quando ocorre um erro.

Por que usar venv? Para isolar as bibliotecas e poder reinstalá-las com requirements.txt.

Os dados estão no Supabase? Só afirmem isso depois da auditoria e de conferir POST/GET com os mesmos IDs no banco remoto. Os testes registrados da preparação usaram SQLite.

Tem inteligência artificial treinada? Esta etapa é Ciência de Dados descritiva. Não há modelo preditivo treinado ou previsão operacional de chegada.

São 180 viagens? Não. São 180 observações, sem identificador de viagem completa.

Os dados persistem? O cadastro confirma uma transação no banco; uma consulta posterior retornou os registros. Testar persistência após reiniciar é uma verificação adicional e não foi confirmado explicitamente nas capturas recebidas.

O site está integrado? O dashboard servido por esta API usa suas rotas. O frontend React/Vite da escola e o site V2 são versões distintas; não afirmem que os fluxos de passageiro usam esta API sem verificar essa integração.

Plano B: se houver falha na hora

• Conexão recusada: confira o terminal e o processo antes de mudar código. Deixe a API aberta.

• Porta ocupada: use a instância já iniciada ou encerre apenas seu servidor antigo. Não mate processos desconhecidos.

• Falha no Supabase ou na internet: use as respostas JSON e as capturas já obtidas, dizendo de onde vieram. Não troque banco nem digite credenciais diante da turma.

• Rotas IA ausentes: a cópia escolar não foi atualizada. Mostre os JSONs versionados da validação na nuvem e explique que as rotas novas precisam ser instaladas nessa máquina. Não finja uma execução ao vivo.

• Banco sem registros: os cadastros do roteiro podem ser feitos ao vivo em banco de testes autorizado. Use o ID realmente retornado, sem pressupor 1 ou 2.

• Não use a nota ou a avaliação simulada do frontend como prova de gravação no banco da API.

Fonte das afirmações

Código de `backend_escola`; respostas de Swagger coladas pelo usuário em `backend_escola/evidencias/testes-escola-2026-10-19.json`; testes e HTTP reais da versão integrada em `backend_escola/evidencias/integracao-ds-ia-2026-10-19/`. O roteiro distingue execução na escola, validação na nuvem e funcionalidades simuladas.